Homework Solutions: Lecture 9

Hypothesis Testing, p-Values & Power

Probability & Statistics Course

Problem 01 · Guilty or Not?

Exercise

A jury must decide whether a defendant is guilty. Map the hypothesis-testing framework onto the courtroom:

a) What plays the role of H_0? Of H_1?

b) What is a Type I error in this context? A Type II error?

c) What is the "significance level" α analogous to?

d) In a **criminal** trial, which error is considered worse? What about in a **medical screening** for a serious disease? How does this affect the choice of α?

Solution

(a) Hypotheses. The null hypothesis is the "default" state – the status quo that holds unless the evidence overwhelms it:

$$H_0\colon \text{The defendant is innocent.}$$

The alternative is the claim that requires proof:

$$H_1\colon \text{The defendant is guilty.}$$

This matches the legal principle of "presumed innocent until proven guilty." Just as in statistics we don't reject H_0 unless the evidence is strong enough, a jury doesn't convict unless guilt has been established.

(b) Type I and Type II errors.

	H_0 **true (innocent)**	H_0 **false (guilty)**
Reject H_0 (convict)	Type I error	Correct
Fail to reject (acquit)	Correct	Type II error

- **Type I error** = convicting an innocent person.
- **Type II error** = acquitting a guilty person (they go free).

(c) The significance level α. α is the maximum probability of a Type I error that we are willing to tolerate. In the courtroom, this corresponds to the **standard of evidence** required for conviction.

In criminal law this standard is "beyond a reasonable doubt" – a very high bar. This is equivalent to choosing a very small α: we want $P(\text{convict innocent})$ to be extremely low, even if it means some guilty defendants go free.

(d) Which error is worse? Criminal trial: Type I (convicting an innocent person) is considered far worse. An innocent person loses their freedom; this is irreversible. Society accepts that some guilty people will escape conviction (Type II) to protect the innocent. This means we want α to be **very small** – the evidence must be overwhelming.

Medical screening for a serious disease: Type II (failing to detect a disease that is actually present) is now the worse error. A missed cancer diagnosis can be fatal; a false alarm (Type I) leads to further testing, which is stressful but not deadly. Here we choose a **larger** α (e.g., 0.10 or even 0.20) to increase sensitivity – we'd rather flag healthy people for follow-up than miss sick ones.

Key insight: the "right" α is not always 0.05. It depends on the relative costs of the two types of error. The more catastrophic Type I is relative to Type II, the smaller α should be (and vice versa).

Problem 02 · The Suspicious Coin

Exercise

You flip a coin 100 times and observe 60 heads.

a) State H_0 and H_1 (two-sided).

b) Compute the z-statistic.

c) Find the p-value. At $\alpha = 0.05$, do you reject? At $\alpha = 0.01$?

d) Build a 95% CI for p. Verify that your test decision matches whether $p_0 = 0.5$ lies inside the CI.

Solution

(a) Hypotheses.

$$H_0 : p = 0.5 \qquad \text{vs} \qquad H_1 : p \neq 0.5$$

This is two-sided: we're testing whether the coin is fair, without specifying a direction of bias.

(b) z-statistic. For a proportion test, the test statistic uses the **null value** p_0 in the standard error (not the sample proportion $\hat{p}$). This is because under H_0 we know the exact variance:

$$\text{SE}_0 = \sqrt{\frac{p_0(1-p_0)}{n}} = \sqrt{\frac{0.5 \times 0.5}{100}} = \sqrt{\frac{0.25}{100}} = \sqrt{0.0025} = 0.05.$$

The z-statistic measures how many standard errors the observed proportion is from the null:

$$z = \frac{\hat{p} - p_0}{\text{SE}_0} = \frac{0.60 - 0.50}{0.05} = \frac{0.10}{0.05} = \boxed{2.0}.$$

Interpretation: we observed a proportion that is 2 standard errors above what we'd expect if the coin were fair.

Common mistake: using $\hat{p} = 0.60$ instead of $p_0 = 0.5$ in the SE. This would give SE $= \sqrt{0.60 \times 0.40/100} = 0.049$, yielding $z = 2.04$ – close in this case, but conceptually wrong. The test asks "how extreme is the data under H_0?", so the SE must be computed under H_0.

(c) p-value. Since this is a two-sided test, the p-value is the probability of getting a z value at least as extreme in *either* direction:

$$p\text{-value} = P(|Z| \geq 2.0) = 2 \cdot P(Z \geq 2.0).$$

From the standard normal table: $P(Z \geq 2.0) = 1 - \Phi(2.0) = 1 - 0.9772 = 0.0228$.

$$p\text{-value} = 2 \times 0.0228 = \boxed{0.0456}.$$

Decisions:

- At $\alpha = 0.05$: $p = 0.0456 < 0.05$, so we **reject** H_0. There is significant evidence the coin is biased.
- At $\alpha = 0.01$: $p = 0.0456 > 0.01$, so we **fail to reject** H_0. The evidence is not strong enough at this stricter level.

Notice how the conclusion depends entirely on the choice of α. The data hasn't changed – only our threshold for "convincing" has.

(d) 95% CI and consistency check. Why $\hat{p}$ now, not p_0? In part (b) we used p_0 in the SE because we were asking "how likely is this data *if H_0 is true*?" – so we computed variability under H_0. For the CI, we are *estimating* the true p without assuming any particular value, so we plug in our best estimate $\hat{p}$. This is a common source of confusion – keep the two purposes straight.

For the CI we use the **sample proportion** in the SE (this is the Wald CI):

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} = \sqrt{\frac{0.60 \times 0.40}{100}} = \sqrt{0.0024} = 0.04899.$$

$$\text{CI} = \hat{p} \pm z_{0.025} \cdot \text{SE}(\hat{p}) = 0.60 \pm 1.96 \times 0.04899 = 0.60 \pm 0.0960.$$

$$\boxed{\text{CI} = (0.504,\ 0.696)}.$$

Consistency check: $p_0 = 0.5$ is **outside** the CI (the lower bound is $0.504 > 0.5$). This is consistent with rejecting H_0 at $\alpha = 0.05$.

This is not a coincidence. For two-sided tests about a single parameter, there is an exact duality: rejecting $H_0 \colon p = p_0$ at level α is equivalent to p_0 falling outside the $(1-\alpha)$ CI. They are the same test expressed in two ways.

Note: the CI barely excludes 0.5 – it's right on the edge, which is consistent with the p-value being barely below 0.05.

Problem 03 · p-Value Misconceptions

Exercise

A study reports $p = 0.03$. For each statement below, say TRUE or FALSE and briefly explain.

Solution

(a) "There is a 3% probability that H_0 is true." **FALSE.** This is the single most common misinterpretation of p-values.

The p-value is $P(\text{data this extreme} \mid H_0 \text{ true})$, **not** $P(H_0 \text{ true} \mid \text{data})$. These are very different quantities – confusing them is sometimes called the "prosecutor's fallacy" or "transposing the conditional."

To see why they differ, consider an analogy: $P(\text{spots} \mid \text{dalmatian}) = 1$, but $P(\text{dalmatian} \mid \text{spots}) \ll 1$ (leopards, ladybugs, and many other things have spots). Conditioning in one direction doesn't tell you the probability in the other direction without Bayes' theorem and a prior.

To compute $P(H_0 \text{ true} \mid \text{data})$ you would need Bayes' theorem, which requires a prior probability $P(H_0)$ – something the frequentist framework does not provide.

(b) "If we reject H_0, there is a 3% chance we made an error." **FALSE.** The p-value is **not** the probability of a Type I error for this specific decision.

The significance level α controls the long-run Type I error rate: if we always reject when $p < \alpha$, then across many studies where H_0 is true, we will be wrong α fraction of the time. But for *this particular study*, either H_0 is true or it isn't – there is no probability involved (in the frequentist view).

Put differently: $\alpha = 0.05$ means 5% of studies where H_0 is true will produce a false rejection. The p-value of 0.03 from *this* study doesn't tell us the probability that *this specific* rejection is an error.

Concrete example: imagine 100 labs each test a different useless supplement (H_0 is true for all 100). About 5 will get $p < 0.05$ and publish a "significant" result. When you read one of those papers reporting $p = 0.03$, the probability you're reading a false positive is not 3% – it's close to 100%, because *all* the supplements are useless. The p-value alone cannot tell you this; you also need to know how many hypotheses were tested. (This connects directly to Problem 04.)

(c) "If we repeated the study many times, we would get $p < 0.05$ at least 97% of the time." **FALSE.** This confuses the p-value with the power of the study.

- If H_0 is **true**: p-values are uniformly distributed on $[0, 1]$, so $P(p < 0.05) = 0.05$. We would get $p < 0.05$ only 5% of the time, not 97%.
- If H_0 is **false**: the replication rate depends on the **power** of the study (which depends on the true effect size and sample size). A single observed $p = 0.03$ does not tell you the power.

In fact, many studies with $p \approx 0.03$ turn out to be underpowered and fail to replicate – this is one driver of the "replication crisis."

(d) "The probability of observing data this extreme (or more), assuming H_0 is true, is 0.03." **TRUE.** This is the correct definition of the p-value:

$$p\text{-value} = P(\text{test statistic} \geq t_{\text{obs}} \mid H_0).$$

(For a two-sided test, "this extreme" means in either tail.)

(e) "The effect is practically important." **FALSE.** Statistical significance $\neq$ practical significance.

A large enough sample can make an arbitrarily small effect statistically significant. For example, a drug that lowers blood pressure by 0.1 mmHg might achieve $p < 0.001$ with $n = 100{,}000$ patients, but that effect is clinically meaningless.

Conversely, a practically important effect might fail to reach significance if the sample is too small.

Always report effect sizes (and confidence intervals) alongside p-values. The p-value tells you whether an effect is distinguishable from zero; the effect size tells you whether it matters.

Problem 04 · The p-Hacking Experiment

Exercise

Simulate the multiple-testing disaster in Python:

a) Generate 20 independent datasets, each containing two groups of $n = 30$ drawn from the same $N(0, 1)$ (so H_0 is true for all 20). Run a two-sample t-test on each pair. How many give $p < 0.05$?

b) Repeat the entire experiment 1,000 times. In what fraction of repetitions do you find at least one significant result ($p < 0.05$)?

c) Compare with the theoretical answer: $1 - (1 - 0.05)^{20} \approx ?$

d) Take one of the runs where several tests are "significant." Apply Bonferroni and Benjamini–Hochberg corrections. How many remain significant after each?

Solution

See the accompanying Jupyter notebook (`hw_09_solutions.ipynb`) for all code and plots.

(a) One round of 20 tests. Each test compares two groups of 30 values, both drawn from $N(0, 1)$. Since H_0 is true for every pair, any "significant" result is a false positive.

Under H_0, each t-test has a 5% chance of yielding $p < 0.05$. With 20 independent tests, we expect $20 \times 0.05 = 1$ false positive on average.

In a typical run (seed 509), we observe 0 "significant" results out of 20 – we got lucky. But this varies from run to run – sometimes 1, sometimes 2 or 3.

(b) Fraction with at least one significant result. Running the experiment 1,000 times, we find that approximately **64%** of repetitions produce at least one "significant" result – even though nothing is real.

(c) Theoretical answer. If the 20 tests are independent, each with $P(\text{reject}) = 0.05$ under H_0:

$$P(\text{at least one reject}) = 1 - P(\text{none reject}) = 1 - (1 - 0.05)^{20} = 1 - 0.95^{20}.$$

Computing:

$$0.95^{20} = 0.3585, \qquad 1 - 0.3585 = \boxed{0.6415}.$$

So there is a **64.15%** chance of finding at least one "significant" result from 20 tests – even when nothing is going on. This is the multiple testing problem in a nutshell: run enough tests, and you're almost guaranteed to find "something."

The simulation result ($\approx 64\%$) matches this theoretical value closely, confirming that the t-tests are behaving as expected under the null.

(d) Corrections. **Bonferroni correction** controls the family-wise error rate (FWER). It divides the significance threshold by the number of tests:

$$\alpha_{\text{Bonf}} = \frac{0.05}{20} = 0.0025.$$

A result is significant only if $p < 0.0025$. In a typical run where 1–2 of the 20 p-values are below 0.05, most (or all) will be above 0.0025, so **0 tests survive** Bonferroni correction.

Bonferroni is conservative: it guarantees $P(\text{any false positive}) \leq 0.05$, but at the cost of reduced power. If some of the 20 tests had real effects, Bonferroni might miss them.

Benjamini–Hochberg (BH) correction controls the false discovery rate (FDR) instead of the FWER. The procedure:

1. Sort the 20 p-values: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(20)}$.
2. For each rank k, compute the threshold $\frac{k}{20} \times 0.05$.
3. Find the largest k such that $p_{(k)} \leq \frac{k}{20} \times 0.05$.
4. Reject all tests with $p_{(i)} \leq p_{(k)}$.

In a typical run with all H_0 true and 1 significant result around $p \approx 0.03$, the smallest threshold is $\frac{1}{20} \times 0.05 = 0.0025$. Since $0.03 > 0.0025$, BH also rejects **0 tests** in this case.

When there are real effects mixed in with nulls, BH is substantially more powerful than Bonferroni while still controlling the expected proportion of false discoveries at $\leq 5\%$.

```
import numpy as np
from scipy import stats

np.random.seed(509)
n_tests, n_per_group = 20, 30

# (a) One round
pvals = []
for _ in range(n_tests):
    g1 = np.random.normal(0, 1, n_per_group)
    g2 = np.random.normal(0, 1, n_per_group)
    _, p = stats.ttest_ind(g1, g2)
    pvals.append(p)

sig = sum(p < 0.05 for p in pvals)
print(f"Significant: {sig} / {n_tests}")

# (b) Repeat 1000 times
n_reps = 1000
any_sig = 0
for _ in range(n_reps):
```

```
    ps = []
    for _ in range(n_tests):
        g1 = np.random.normal(0, 1, n_per_group)
        g2 = np.random.normal(0, 1, n_per_group)
        _, p = stats.ttest_ind(g1, g2)
        ps.append(p)
    if min(ps) < 0.05:
        any_sig += 1

print(f"At least one sig: {any_sig/n_reps:.3f}")

# (c) Theoretical
print(f"Theoretical: {1 - 0.95**20:.4f}")

# (d) Corrections
from statsmodels.stats.multitest import multipletests

pvals = np.array(pvals)
_, p_bonf, _, _ = multipletests(pvals, alpha=0.05, method='bonferroni')
_, p_bh, _, _ = multipletests(pvals, alpha=0.05, method='fdr_bh')

print(f"Bonferroni survivors: {sum(p_bonf < 0.05)}")
print(f"BH survivors: {sum(p_bh < 0.05)}")
```

Problem 05 · Build Your Own Permutation Test

Exercise

Treatment group: [5.2, 6.1, 7.3, 5.8, 6.9]. Control group: [4.1, 3.9, 5.0, 4.5, 4.3].

a) Compute the observed difference in means $\bar{X}_T - \bar{X}_C$.

b) Under H_0 (no treatment effect), all $\binom{10}{5} = 252$ ways to split the 10 values into two groups of 5 are equally likely. Enumerate all 252 permutations and compute $\bar{X}_T^* - \bar{X}_C^*$ for each.

c) What fraction of permutations produce a difference $\geq$ the observed value? This is your exact one-sided p-value.

d) Compare with the Welch t-test p-value (`scipy.stats.ttest_ind`). Do they agree?

Solution

(a) Observed difference in means. Treatment:

$$\bar{X}_T = \frac{5.2 + 6.1 + 7.3 + 5.8 + 6.9}{5} = \frac{31.3}{5} = 6.26.$$

Control:

$$\bar{X}_C = \frac{4.1 + 3.9 + 5.0 + 4.5 + 4.3}{5} = \frac{21.8}{5} = 4.36.$$

$$\boxed{\bar{X}_T - \bar{X}_C = 6.26 - 4.36 = 1.90}.$$

(b) Enumerating all permutations. The key idea: under H_0, the treatment labels are arbitrary. If the treatment has no effect, each observation would have been the same regardless of which group it was assigned to. So all $\binom{10}{5} = 252$ ways to assign 5 of the 10 observations to the "treatment" group are equally likely.

The pooled data is:

$$\{3.9,\ 4.1,\ 4.3,\ 4.5,\ 5.0,\ 5.2,\ 5.8,\ 6.1,\ 6.9,\ 7.3\}.$$

The total sum is 53.1. For any split into two groups of 5 with "treatment" sum S_T:

$$\bar{X}_T^* - \bar{X}_C^* = \frac{S_T}{5} - \frac{53.1 - S_T}{5} = \frac{2S_T - 53.1}{5}.$$

We enumerate all 252 subsets of size 5 using `itertools.combinations` and compute the difference for each. See the notebook for the full enumeration and histogram.

(c) Exact one-sided p-value. We need the fraction of permutations with difference ≥ 1.90, i.e., $S_T \geq 31.3$.

Note that the observed treatment group $\{5.2, 5.8, 6.1, 6.9, 7.3\}$ has $S_T = 31.3$. This is the **five largest values** in the dataset. Any other subset of size 5 must include at least one value smaller than 5.2, replacing one of these, so its sum would be strictly less than 31.3.

Therefore, exactly **1 out of 252** permutations produces a difference ≥ 1.90:

$$\boxed{p\text{-value} = \frac{1}{252} \approx 0.00397}.$$

This is very strong evidence against H_0: the observed split is literally the most extreme possible.

Common mistake: computing a two-sided p-value by counting $|\bar{X}_T^* - \bar{X}_C^*| \geq 1.90$. The problem asks for a *one-sided* test ("difference $\geq$ the observed value"). For a two-sided permutation test you would also count the left tail (differences ≤ -1.90), giving $p = 2/252 \approx 0.0079$.

(d) Comparison with Welch t-test. Running `scipy.stats.ttest_ind(treatment, control, equal_var=False)`:

- $t = 4.50$, two-sided $p = 0.0043$.
- One-sided $p = 0.0043/2 = 0.0022$.

The Welch t-test gives $p \approx 0.002$ (one-sided), while the permutation test gives $p \approx 0.004$. Both are highly significant and agree on the conclusion: the treatment group has a significantly higher mean.

The slight difference arises because:

- The t-test assumes normally distributed data and uses a continuous t-distribution.
- The permutation test makes no distributional assumptions and uses the exact discrete distribution of all possible label assignments.

With only 5 observations per group, the permutation test is arguably more trustworthy since it doesn't rely on the normal approximation. As sample sizes grow, both approaches converge.

```python
import numpy as np
from itertools import combinations
from scipy import stats

treatment = np.array([5.2, 6.1, 7.3, 5.8, 6.9])
control = np.array([4.1, 3.9, 5.0, 4.5, 4.3])

# (a) Observed difference
obs_diff = treatment.mean() - control.mean()
print(f"Observed diff = {obs_diff:.2f}")

# (b) All permutations
pooled = np.concatenate([treatment, control])
diffs = []
for idx in combinations(range(10), 5):
    grp1 = pooled[list(idx)]
    grp2 = pooled[[i for i in range(10) if i not in idx]]
    diffs.append(grp1.mean() - grp2.mean())

diffs = np.array(diffs)
print(f"Number of permutations: {len(diffs)}")

# (c) Exact one-sided p-value
p_perm = np.mean(diffs >= obs_diff)
print(f"Permutation p-value = {p_perm:.5f} ({np.sum(diffs >= obs_diff)}/252)")

# (d) Welch t-test
t_stat, p_welch = stats.ttest_ind(treatment, control, equal_var=False)
print(f"Welch t = {t_stat:.3f}, two-sided p = {p_welch:.4f}")
print(f"One-sided p = {p_welch/2:.4f}")
```